

WEEK 5: PAYMENT TRANSACTION MANAGEMENT SYSTEM

Scenario

The e-commerce company needs to maintain payment information for customer orders. The system should record payment details, track transaction status, manage successful and failed payments, and generate payment-related business reports.

The **ArtHub** system uses a Payment Transaction Module connected with the existing Customer and Order Management modules. The Payment table stores the payment details associated with each order.

1. Create Payment Table

The **Payment** entity is used to store customer payment transaction details.

The Payment table contains the following attributes:

Attribute	Description
Payment_ID	Unique payment identifier
Order_ID	Order reference
Payment_Date	Date of transaction
Payment_Mode	UPI, Card, Net Banking or Cash
Payment_Status	Success, Failed or Pending
Transaction_Amount	Paid amount

For ArtHub, Payment_ID was created as VARCHAR2 so that payment identifiers such as **P1, P2, P3** can be used. The Order_ID is maintained consistently with the existing Orders table.

```
SQL> CREATE TABLE Payment (Payment_ID VARCHAR2(10), Order_ID VARCHAR2(10), Payment_Date DATE, Payment_Mode VARCHAR2(20), Payment_Status VARCHAR2(20), Transaction_Amount NUMBER(10,2));  
Table created.
```

2. Establish Relationship with Orders

The Payment table is connected to the existing **Orders** table through Order_ID.

The relationship ensures that each payment is associated with a valid order in the ArtHub system.

Relationship:

Orders → Payment

The Order_ID in the Payment table acts as a **Foreign Key** referencing the Order_ID in the Orders table.

3. Apply Keys and Constraints

The following keys and constraints were applied to maintain data integrity:

Primary Key

Payment_ID was defined as the **Primary Key** of the Payment table.

```
SQL> ALTER TABLE Payment ADD CONSTRAINT pk_payment PRIMARY KEY (Payment_ID);  
Table altered.
```

Foreign Key

Order_ID was defined as a **Foreign Key** referencing the Orders table.

```
SQL> ALTER TABLE Payment ADD CONSTRAINT fk_payment_order FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID);  
Table altered.
```

NOT NULL Constraints

The following attributes were made mandatory:

- Order_ID
- Payment_Date
- Payment_Mode
- Payment_Status

```
SQL> ALTER TABLE Payment MODIFY Payment_Mode VARCHAR2(20) NOT NULL;  
Table altered.  
  
SQL> ALTER TABLE Payment MODIFY Payment_Status VARCHAR2(20) NOT NULL;  
Table altered.
```

CHECK Constraint

Transaction_Amount was restricted to values greater than zero.

These constraints ensure that valid and meaningful payment information is stored in the database.

```
SQL> ALTER TABLE Payment ADD CONSTRAINT chk_payment_amount CHECK (Transaction_Amount > 0);  
Table altered.
```

4. Store Payment Details

Payment details were inserted for the existing orders in the ArtHub system.

Payment identifiers were assigned as **P1 to P7**, corresponding to orders **O1 to O7**.

The payment records contain:

- Payment ID
- Order ID
- Payment date
- Payment mode
- Payment status
- Transaction amount

The payment records were then verified and committed to the database.

```
SQL> INSERT INTO Payment (Payment_ID, Order_ID, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount) VALUES ('P1', '01', TO_DATE('20-09-2026','DD-MM-YYYY'), 'UPI', 'Completed', 5000);
1 row created.

SQL> INSERT INTO Payment (Payment_ID, Order_ID, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount) VALUES ('P2', '02', TO_DATE('21-09-2026','DD-MM-YYYY'), 'Card', 'Completed', 5200);
1 row created.

SQL> INSERT INTO Payment (Payment_ID, Order_ID, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount) VALUES ('P3', '03', TO_DATE('21-09-2026','DD-MM-YYYY'), 'UPI', 'Completed', 1800);
1 row created.

SQL> INSERT INTO Payment (Payment_ID, Order_ID, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount) VALUES ('P4', '04', TO_DATE('22-09-2026','DD-MM-YYYY'), 'Net Banking', 'Pending', 2200);
1 row created.

SQL> INSERT INTO Payment (Payment_ID, Order_ID, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount) VALUES ('P6', '06', TO_DATE('23-09-2026','DD-MM-YYYY'), 'UPI', 'Completed', 4000);
1 row created.

SQL> INSERT INTO Payment (Payment_ID, Order_ID, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount) VALUES ('P5', '05', TO_DATE('22-09-2026','DD-MM-YYYY'), 'Card', 'Completed', 3500);
1 row created.

SQL> INSERT INTO Payment (Payment_ID, Order_ID, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount) VALUES ('P7', '07', TO_DATE('24-09-2026','DD-MM-YYYY'), 'Card', 'Pending', 2800);
1 row created.

SQL> commit;
```

```
SQL> SELECT * FROM Payment;
```

PAYMENT_ID	ORDER_ID	PAYMENT_DATE	PAYMENT_MODE
P1	01	20-SEP-26	UPI
Completed		5000	
P2	02	21-SEP-26	Card
Completed		5200	
P3	03	21-SEP-26	UPI
Completed		1800	
P4	04	22-SEP-26	Net Banking
Pending		2200	
P6	06	23-SEP-26	UPI
Completed		4000	
P5	05	22-SEP-26	Card
Completed		3500	
P7	07	24-SEP-26	Card
Pending		2800	

7 rows selected.

5. Manage Successful and Failed Transactions

The Payment Transaction Management System is used to monitor the status of payment transactions. The system supports checking successful payments, identifying failed transactions, and counting successful and failed payments.

Task 1: Display All Successful Payments

This report displays all payment transactions whose status is **Completed/Successful**.

It helps the organization identify payments that have been successfully processed.

```
SQL> SELECT * FROM Payment WHERE Payment_Status = 'Completed';
```

PAYMENT_ID	ORDER_ID	PAYMENT_DATE	PAYMENT_MODE
P1 Completed	01	20-SEP-26	UPI
P2 Completed	02	21-SEP-26	Card
P3 Completed	03	21-SEP-26	UPI

PAYMENT_ID	ORDER_ID	PAYMENT_DATE	PAYMENT_MODE
P6 Completed	06	23-SEP-26	UPI
P5 Completed	05	22-SEP-26	Card

Task 2: Find Failed Transactions

This report identifies payment transactions having a **Failed** status.

It can be used to monitor unsuccessful payment attempts.

```
SQL> SELECT * FROM Payment WHERE Payment_Status = 'Failed';  
no rows selected
```

Task 3: Count Total Successful and Failed Payments

This report provides the total number of successful and failed payment transactions separately.

This helps in understanding the overall payment transaction status.

```
SQL> SELECT Payment_Status, COUNT(*) AS Total_Payments FROM Payment WHERE Payment_Status IN ('Completed','Failed') GROUP BY Payment_Status;
```

PAYMENT_STATUS	TOTAL_PAYMENTS
Completed	5

6. Analyze Payment Methods Used by Customers

Payment methods are analyzed to understand how customers make payments in the Arthub system. The report includes the number of UPI transactions, number of card payments, and the most preferred payment method.

Report 1: Payment Mode Analysis

UPI Transactions

The number of transactions made using **UPI** is calculated.

This helps determine the usage of UPI as a payment method.

```
SQL> SELECT COUNT(*) AS UPI_Transactions FROM Payment WHERE Payment_Mode = 'UPI';
```

UPI_TRANSACTIONS
3

Card Payments

The number of transactions made using **Card** is calculated.

This helps identify the usage of card payments among customers.

```
SQL> SELECT COUNT(*) AS Card_Payments FROM Payment WHERE Payment_Mode = 'Card';
```

CARD_PAYMENTS
3

Most Preferred Payment Method

The payment modes are grouped and their transaction counts are compared.

The payment method with the highest number of transactions is identified as the **most preferred payment method**.

```
SQL> SELECT Payment_Mode, COUNT(*) AS Total_Transactions FROM Payment GROUP BY Payment_Mode HAVING COUNT(*) = (SELECT MAX(COUNT(*)) FROM Payment GROUP BY Payment_Mode);
```

PAYMENT_MODE	TOTAL_TRANSACTIONS
Card	3
UPI	3

7. Revenue Analysis

Revenue analysis is performed using the transaction amounts stored in the Payment table. It helps the organization understand the total revenue and the contribution of different payment methods.

Report 2: Revenue Analysis

Total Revenue Generated

The total transaction amount from all payment records is calculated.

This represents the total revenue generated through the recorded payment transactions.

```
SQL> SELECT SUM(Transaction_Amount) AS Total_Revenue FROM Payment;
```

TOTAL_REVENUE
24500

Revenue by Payment Method

The transaction amounts are grouped according to payment mode.

This report shows the revenue generated through each payment method such as UPI, Card and Net Banking.

```
SQL> SELECT Payment_Mode, SUM(Transaction_Amount) AS Revenue FROM Payment GROUP BY Payment_Mode ORDER BY Revenue DESC;
```

PAYMENT_MODE	REVENUE
Card	11500
UPI	10800
Net Banking	2200

Average Transaction Amount

The average of all transaction amounts is calculated.

This provides the average value of a payment transaction in the ArtHub system.

```
SQL> SELECT AVG(Transaction_Amount) AS Average_Transaction_Amount FROM Payment;
```

AVERAGE_TRANSACTION_AMOUNT
3500

8. Customer Payment History

Customer Payment History is generated by connecting the **Customer**, **Orders**, and **Payment** tables.

The report displays the following information:

- Customer Name
- Order ID
- Payment Mode
- Transaction Amount
- Payment Status

This report helps track the payment details associated with each customer's orders.

9. Payment Analysis

The payment analysis provides an overall view of the transactions recorded in the ArtHub system.

The analysis helps identify:

- Successful and failed payment transactions

- Usage of different payment methods
- Most preferred payment method
- Total revenue generated
- Revenue based on payment method
- Average transaction amount
- Payment history of customers

The results can help the organization understand customer payment behavior and monitor payment-related activities.

Conclusion

The **Payment Transaction Management System** was successfully implemented for the ArtHub project. The Payment table was created and connected with the existing Orders table using appropriate keys and constraints.

Payment details were stored and analyzed based on transaction status, payment method, revenue, and customer payment history. The system provides useful payment-related reports that help in monitoring transactions and understanding payment patterns.